Deep Implicit Layers for Smooth Implied Volatility Surface Interpolation*

Rafael Zimmer
Institute of Mathematical Science and Computing
University of São Paulo - Brazil
rafael.zimmer@usp.br

***Abstract*—We implement a neural network architecture based on deep implicit layers with the objective of optimizing the parameters of multiple SABR (stochastic alpha, beta, rho) models for interpolation and extrapolation of implied volatility surfaces in the context of financial options. The parameter values for the models are trained on daily data points and used for generating smooth interpolated points in the surface. The motivation behind our approach is to reduce arbitrage opportunities when generating smooth interpolation and extrapolation of the surface, i.e., making it infinitely differentiable at all points while still matching existing contracts. The proposed architecture solves the problem of optimizing for the α, ρ, ν parameters while minimizing statistical arbitrage opportunities for observed contracts within the generated surface. Our methodology includes using historical observed values of listed options as well as the calculated implied volatility for these contracts. The innate ability of transformer architectures to process observations of sequences with non-fixed lengths, i.e., for contracts with varying expiration dates is used to choose candidate points that best summarize the daily surfaces. The generated surfaces are then compared to a baseline linear interpolation model.**

***Index Terms*—Neural Networks, Implied Volatility, Transformers, Quantitative Methods, Implicit Layers.**

I. Introduction

A. Brief Introduction to Options and Implied Volatility

Option contracts are one of the most important types of derivative contracts in the context of financial markets. A derivative is a contract traded with respect to another asset (also called the underlying asset) and an option contract allows the buyer of the contract to sell or buy the specified asset from the seller at a predefined future date. The pricing equations for these derivatives, called the Black-Scholes model, require knowing the values for the time to maturity of the contract, the price of the underlying asset to be traded, the price of the asset to be traded and the market's risk-free rate at the time [1]. An additional variable, called the implied volatility (*IV* or σ_{iv}) is also required for the pricing of these contracts. This value refers to the market's expected value for the volatility of the asset price up until the time of maturity.

Having access to the implied volatility values allows investors to make sales at the prices consistent with the current expected contract price, that is, the no-arbitrage price according to the market expectancy. However, the *IV* values corresponding to all possible strike prices and maturities are not directly observable in the market. Ensuring a way to obtain these values with confidence and readily available data in the market is a crucial task for quantitative analysts [12]. Usual strategies for estimating these values daily are predominantly based on linear or spline interpolation algorithms [9] or stochastic differential equation models [10] that use numerical methods to approximate the values from existing listed contracts. Both methods try to approximate the correct value for the implied volatility for each pair of time to maturity and strike price. This approach allows visualizing the model results in the form of a surface, where the **XY** axes are the maturity and strikes pairs and the **Z** axis is the *IV*. This surface, called the **implied volatility surface** (IVS), noted as $\mathcal{S}$ is therefore a function of the maturity time t and the strike price K,

$$\mathcal{S} \coloneqq f(t, K), \quad \text{where } f\colon \mathbb{R}^2 \to \mathbb{R}. \tag{1}$$

Ensuring accurate predictions of the IVS is crucial, as even small errors can lead to incorrect pricing and consequently large losses of profit or even negative return [8] given the leveraging capabilities of options when compared to non-derivative securities. Classic models such as linear interpolation, polynomial fitting, and *splines* not only lack flexibility for extrapolation, but also generate either non-smooth surfaces, arbitrage opportunities, or both [8].

B. Contributions of the Presented Work

In this paper, we propose an approach to address the challenges of interpolation and extrapolation of the IVS, focusing on reducing arbitrage opportunities. The chosen method uses a stochastic differential equation solution approach, specifically the *Stochastic Alpha, Beta, Rho* (*SABR*) model. The parameters α, ρ, ν of the *SABR* model are obtained through the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton non-linear optimization algorithm [2, 4, 5, 19]. This step is commonly performed using only a subset of the points observed daily [13], due to the fact that using all points introduces various outliers that do not match the real prices of the derivative, which in turn increases the variance of the final model. To solve this difficulty, the points chosen for the optimization step are often manually filtered according to criteria set by human experts [6], and only a candidate subset of the original points is used. Rather than manually selecting these candidate points, the use of models capable

*This work was supported by the São Paulo State Research Support Foundation (FAPESP).

of receiving point clouds and returning a score, such as transformers, to select the best points automatically has been proposed by Hagan et al. [6] instead. The main contribution of this research will be the adaptation of *neural implicit layers*, a physics-inspired variation of fully connected linear layers that outputs a solution to a fixed point equation, to be used as the output layer to a transformer architecture. This approach is an implicit fixed-point solver variation for the scoring model proposed by Kim et al. [13], aiming to enforce a zero arbitrage condition in the optimization process. This type of layer has efficiently been shown to perform well representing smooth surface densities and demonstrated state-of-the-art performance in function fitting tasks similar to the one presented here [11, 15], all while being computationally faster for solving interpolation tasks than linear layer models with similar performance [15]. Additionally, transformers have also been shown to perform better at handling unordered point clouds over time compared to recurrent neural layers [14].

C. The Transformer Architecture

The aforementioned *SABR* model is a stochastic volatility model used to obtain smooth surfaces, i.e., continuous and infinitely differentiable at all points [7]. In the context of this work, a score will be assigned for each candidate point of the daily listed options contracts, called the *summarized suitability* score (*SS* score) by Kim et al. [13]. This score corresponds to the suitability of the candidate point to summarize the characteristics of the generated surface, i.e., how feasible it is to reduce the number of candidates when optimizing the values for one of the parameters of the SABR model without altering the resulting surface shape or its smoothness[1]. In summary, the score is used to select the highest valued points, and therefore the most representative of the surface, removing those that might introduce arbitrage in the optimization process for each of the *SABR* models [13] (a separate model is fitted for each unique strike price in a given day). Section II-A covers the details of fitting the *SABR* model at each strike price.

D. Fixed-Point problems

Differently from the usual function approximation tasks, a fixed point problem is instead defined by conditions that the output must satisfy. For explicit functions, a given output z is computed directly from the input x using a function f, such that $z = f(x)$. In contrast, an implicit function g is such that it depends on both the input x and the expected output z, where z is a solution for the equation:

$$g(x, z) = 0 \tag{2}$$

Formulating the problem as a fixed point equation can be used for a multitude of problems, including finding fixed points in recurrent networks, solutions to differential equations leading to Neural ordinary differential equations, and several physics optimization problems [3]. The implicit formulation offers several practical advantages, primarily the separation of the solution method from the layer definition which can be useful in maintaining interpretability of the resulting model[11] and as a way to enforce constraints in the optimization process.

Moreover, the resulting implicit layer offers significant benefits specifically in deep learning and automatic differentiation frameworks [15, 18]. Automatic differentiation methods store the entire computation graph, which can be memory-intensive. With implicit layers, however, the implicit function theorem allows computing gradients directly at the solution point and therefore storing intermediate variables becomes redundant [18]. This improves memory efficiency, making implicit layers particularly advantageous when substituting existing layers within large deep learning models [16].

E. Anderson Acceleration for Implicit Layers

The Anderson Acceleration (AA) method is a technique used to accelerate the convergence of fixed-point iterations. It is particularly useful when the fixed-point iteration is slow to converge or when the fixed-point iteration is used in a deep learning context. The method is based on the idea of extrapolating the current fixed-point iteration using a linear combination of the previous iterations. For this work, we use a Equilibrium Model variation of the Anderson Acceleration method, which is particularly useful for implicit layers in neural networks. The AA method is used to solve the fixed-point equation within the layer, by extrapolating the current fixed-point iteration using a linear combination of the previous iterations. The AA method is particularly useful for implicit layers in neural networks, as it allows for faster convergence of the fixed-point iteration. Our proposed architecture uses the AA method to solve the fixed-point equation in the implicit layer, according to the Algorithm 1 for our implementation of the AA method.

The final deep equilibrium model used within the implicit layer uses the Anderson Acceleration algorithm to solve the fixed-point equation, and the backpropagation algorithm is used to find the weights that minimize the error function of the model. The solution to the fixed-point equation, is then used as the output of the layer, as shown by Algorithm 2.

[1]The use of a per-point score is similar to what the principal components of a collection of points is when using the Principal Component Analysis (PCA) technique.

Algorithm 1 Equilibrium Model with Anderson Acceleration

Require: Function f, initial point x_0, memory size m, damping factor λ, maximum iterations K, tolerance ϵ, mixing factor β

Initialize $X_0 = x_0, X_1 = F_0 = f(x_0), F_1 = f(F_0)$
for $k = 2$ to K **do**
  $j \leftarrow \min(k, m)$
  $G = F_{0,\ldots,j} - X_{0,\ldots,j}$
  $H = GG^\top + \lambda I$
  Solve $H^{-1} \cdot y = \alpha$
  $\alpha \leftarrow \alpha_{1,\ldots,n+1}$
  $X_k \leftarrow \beta\alpha^\top F_0 + (1-\beta)\alpha^\top X_0$
  $F_k \leftarrow f(X_k)$
  Compute residual:

$$\text{residual} = \frac{\|F_k - X_k\|}{\|F_k\| + 10^{-5}}$$

  if residual $< \epsilon$ **then**
    break
  end if
end for
return X_k

Algorithm 2 Implicit layer forward pass with Anderson acceleration (AA).

function IMPLICITLAYER($x, m, \lambda, \epsilon, \beta$)
  function $f(x) \leftarrow W \cdot x + b$
  $x \leftarrow \texttt{AA}(f, x, m, \lambda, \epsilon, \beta)$
  $y \leftarrow x \cdot W^T + b$
  return y
end function

II. Implementation of a Parametetric SABR Model and Implicit Neural Network Architectures

A. Defining the SABR Model Equations

The SABR model is a stochastic differential model, defined by the following equations in which the dynamics of the forward price and the corresponding volatility levels are used to calculate the implied volatility.

- **Alpha** (α): This is the initial volatility level.
- **Beta** (β): The elasticity parameter between the volatility of the asset and its price.
- **Rho** (ρ): Statistical correlation between the asset price and its volatility, also called *spot-volatility correlation*.
- **Volvol** (ν): The volatility of the volatility process itself.

The implied volatility function of the SABR model is then given by:

$$\sigma_{BS}(f, K) = \alpha \frac{(fK)^{\frac{1-\beta}{2}}}{1 + \left(\frac{(1-\beta)^2\rho^2}{24} + \frac{(1-\beta)^2(2-3\rho^2)}{1920}\right)(fK)^{1-\beta}} \tag{3}$$

In the context of this paper, multiple SABR models are fitted daily, one for each unique maturity value at the respective day. Three continuous functions are then used to map the different maturities to the three model parameters (values for β are fixed and remain unchanged). This is done to allow for a smooth transition between the different maturity levels, as using an single SABR model by itself is known to have discontinuities in the implied volatility surface [7, 13].

Given the functions for the parameters $\alpha_t = f_\alpha(t, \mathbf{P})$, $\rho_t = f_\rho(t, \mathbf{Q})$, and $\nu_t = f_\nu(t, \mathbf{R})$, and the encoding variables **P, Q**, and **R** respectively, such that $\mathbf{P} \in \mathbb{R}^5$, $\mathbf{Q} \in \mathbb{R}^4$, and $\mathbf{R} \in \mathbb{R}^4$, the daily parameters for the SABR model are obtained by the following equations for any continuous value of t:

$$f_\alpha(t, \mathbf{P}) = p_0 + \frac{p_3}{p_4}\left(\frac{1 - e^{-p_4 t}}{p_4 t}\right) + \frac{p_1}{p_2}e^{-p_2 t} \tag{4}$$

$$f_\rho(t, \mathbf{Q}) = q_0 + q_1 t + q_2 e^{-q_3 t} \tag{5}$$

$$f_\nu(t, \mathbf{R}) = r_0 + r_1 t^{r_2} e^{r_3 t} \tag{6}$$

The task of calculating the encoding variables **P**, **Q**, and **R** for daily SABR models is then a supervised regression task. As previously mentioned, we use a transformer architecture to approximate the SS scores for each candidate point $\mathcal{S}_i^t \in \mathcal{S}^t$ from the daily observed input. Then, the top $p = 80\%$ candidate points are used as inputs to the BFGS algorithm, targeting the values **P, Q**, and **R** and the objective function being minimizing the error function with respect to the selected candidate set. A list of the most commonly occurring maturity times, and their corresponding implied volatilities are used for the error function calculation, similarly to the selection a human trader would make when selecting the most relevant data points to train the model [7, 13]. For our training process, we use the 7 most common maturities within the entire dataset (as used by Kim et al. [13]).

B. Proposed Encoder Architecture and Error Functions

Transformers are a specific type of neural network architecture capable of capturing complex dependencies in sequential non-fixed length data. Within our proposed architecture, we use a transformer encoder to approximate the *SS* scores for each candidate point $\mathcal{S}_i^t \in \mathcal{S}^t$ from the daily observed input.

1) Details on the Choosen Encoder Architecture: Unlike recurrent neural networks (RNNs) and convolutional neural networks (CNNs), transformers do not rely on a sequential structure to process data. Instead, they use an attention mechanism, which allows each input element (or token) to consider the relationship with all other elements in the same sequence simultaneously. This is achieved through a series of layers using attention heads.

The attention mechanism accepts continuous data and calculates a value for different heads through the value (V), key (K), and query (Q) matrices, resulting in an output weighted by the importance of each element in the sequence [20, 22]. Specifically, for our problem we use the mechanism of self-attention, and therefore the three matrices are equal.

$$\textit{Attention}(Q, K, V) = \textit{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{7}$$

Transformers are classified as machine learning algorithms due to their ability to learn patterns and relationships from data, and are trained using supervised techniques. In the context of our specific architecture, we use only the encoder block and adapt it to perform the regression on the SS scores. The hidden values are passed to fully-connected (FC) layers, and a final implicit fixed-point iteration block is used to obtain the scores as *SS* z-normalized values. The training epochs adjust the layer weights with an error function that minimizes the difference between its predictions and expected z-score values.

2) Details on the Implicit Layer: Using a Deep Equilibrium Model variation for the Implicit Layer, we adapt Equation 2 by using the following fixed point equation instead:

$$\begin{aligned} \mathrm{g}_{\mathbf{W}}(x, z^*) &= 0 \\ \mathrm{z}^* &= f(z^*, x) \end{aligned} \tag{8}$$

where f is our activation function, $\mathbf{W}$ are the layer weights and biases, and $z*$ is the target output of a batched non-normalized Jacobian to be used as the gradient function in our code, given by the equation below. Finally, y are the hidden values as well as the final output scores.

$$\left(\frac{\partial z^*(\cdot)}{\partial(\cdot)}\right)^T y = \left(\frac{\partial f(z^*, x)}{\partial(\cdot)}\right)^T g_{\mathbf{W}}$$

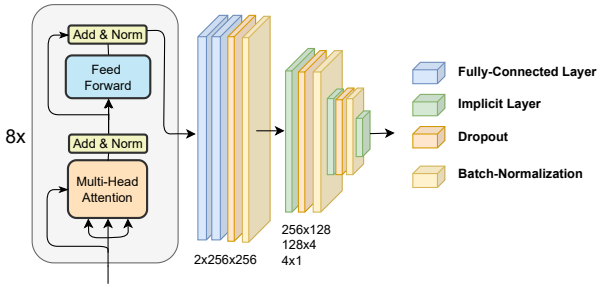

Fig. 1: Visualization of the Transformer Encoder and Implicit Layers used in the proposed architecture.

3) Monte Carlo Estimation of Z-Scores: Instead of using the entire daily candidate set for calculating the encoded parameters **P**, **Q**, and **R**, only a small subset of the candidate set is used to estimate the synthetic target z-scores for the model.

For each data point the mean absolute error (MAE) between the parameter (**P**, **Q**, or **R**) fitted with all data points and the parameter fitted with a smaller percentage of the candidate set is calculated (**P'**, **Q'**, or **R'**) is estimated using Monte Carlo estimation with $k = 1e2$ generated subsets from the original set, where k is the number of subsets and $p = 0.8$ is the percentual size of the subset relative to the daily candidate set.

This estimation using a finite number of subsets is necessary due to computational constraints, as the Monte Carlo estimation is a time-consuming process. Finally, for each training epoch the estimated z-score is normalized and used as target variable for the SS transformer model. Consequentially, if after training the model has converged to a mean low error for the estimated scores, the network parameters are deemed optimized to substitute the Monte Carlo process in estimating the daily SS scores for selecting the candidate set that best captures the daily implied volatility surface.

Algorithm 3 Monte Carlo Estimation of Target Z-Scores

Require: Daily candidate set $\mathcal{S}^t$, percentage $p = 0.8$, number of subsets $k = 1e2$

$\alpha^*, \rho^*, \nu^* \leftarrow$ BFGS($\mathcal{S}^t$)
$\mathrm{SS}_\alpha, \mathrm{SS}_\rho, \mathrm{SS}_\nu \leftarrow \mathbf{0}$
for candidate c in $\mathcal{S}^t$ **do**
  $\mathcal{S}_i^t \leftarrow$ Sample($\mathcal{S}^t$, k, p)
  $\widetilde{\mathbf{P}}, \widetilde{\mathbf{Q}}, \widetilde{\mathbf{R}} \leftarrow$ BFGS($\mathcal{S}_i^t$)
  $\mathrm{SS}_{\alpha,c} \leftarrow \mathrm{MAE}(f_\alpha(t, \widetilde{\mathbf{P}}) - \alpha^*)$
  $\mathrm{SS}_{\rho,c} \leftarrow \mathrm{MAE}(f_\rho(t, \widetilde{\mathbf{Q}}) - \rho^*)$
  $\mathrm{SS}_{\nu,c} \leftarrow \mathrm{MAE}(f_\nu(t, \widetilde{\mathbf{R}}) - \nu^*)$
end for
z-normalize $\mathrm{SS}_\alpha, \mathrm{SS}_\rho, \mathrm{SS}_\nu$

Finally, the values for z-scores are used as the regression target for our model, and the error function is calculated as the root mean squared error (RMSE) between the predicted and target z-scores.

III. Methodology and Results

A. Data Description and Pre-Processing

The data used for training was obtained from historical options data on the SPY asset from January 2021 to December 2022. The dataset contains various columns about traded options, indexed by date, including strike prices, future prices, maturities, implied volatilities, among others. Relevant to our study, the principal columns used were:

- **Underlying**: Price of the underlying asset.
- **Strike**: Option's strike price.
- **Expiration**: Days remaining until the option's expiration.
- **Implied Volatility**: Value for the implied volatility.
- **Date**: Current date, used as the preprocessed set index.

Additionally, two columns were calculated to be used as features for the model:

- **Maturity**: Calculated by dividing *Expiration* by 252 (working days in a year) to normalize the maturity time.
- **Risk-free rate**: Risk-free rate obtained from the Federal Reserve Economic Data (FRED) for the corresponding period.
- **Dividend**: Dividend yield paid quarterly by the SPY index.

B. Data Preprocessing

The data preprocessing involved several essential steps, predominantly performed to allow the use of the data directly in the transformer architecture and subsequent use to generate the surface for historical data. Each of the trained transformers (one for each parameter α, ρ, or ν) takes four dimensions as input, which are:

1) Time to maturity;
2) The actual input value for the parameter (α, ρ, or ν);
3) $\mathbf{1}$ if the parameter was calculated for the current day, $\mathbf{0}$ if it was calculated for a fixed set of maturities (top 7 most frequent maturities) using the previous day fitted parameters;
4) Parameter value calculated using the previous day's $\mathbf{P}$, $\mathbf{Q}$ and $\mathbf{R}$ values for same strike and maturity.

The target variable is the scores generated using all candidate points for the current day, and the model's error function uses the percentage $p = 80\%$ of points and is trained to minimize the quadratic error function of the scores. For training and testing we also observed that the choice of $\beta = 0.5$ performed best at minimizing the error function.

C. Results and Surface Comparison

A baseline linear interpolation implementation for the daily observed points was also used to compare our trained model's performance by interpolating the points internal to a grid of strikes and maturity dates generated for within the boundary pairs for each day, that is, no extrapolation was performed due to the hardships of the linear model to predict values outside the observed range.

Our transformer model was trained on the dataset, with about 68 days and 44k observations total. With an early stopping sensitivity of $\epsilon = 10^{-3}$, the training was performed with a maximum epoch range of 10^3 epochs using the Adam optimizer. For the implicit layer, we used the Anderson Acceleration algorithm with a maximum of 10 iterations and a convergence threshold of 10^{-6}. The training time for the transformer model was approximately 8 minutes and 11 seconds, while the Monte Carlo preprocessing time was around 21 minutes and 5 seconds for the entire dataset. Due to the nature of the implicit layer, a simple batch size of 1 was used to avoid memory issues. The used architecture is a transformer encoder block with 8 attention heads, 2 linear layers with 256 hidden units and 2 implicit layers with 256 and 128 hidden units, respectively. An illustration of the architecture is shown in Figure 1, and a dropout rate of 0.1 was used for the dropout layers.

Model	Test Loss (RMSE)	Train time (Transformer)	Preprocess Time (Monte Carlo)
SST_α	1.21	~4m21s	~7m50s
SST_ρ	0.72	~2m8s	~6m34s
SST_ν	0.75	~1m43s	~6m41s

TABLE I: Model training metrics for each of the three SST models.

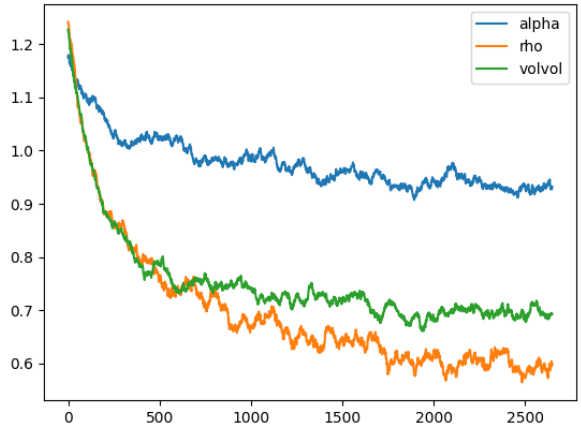

Fig. 2: Historical loss function from on training session data.

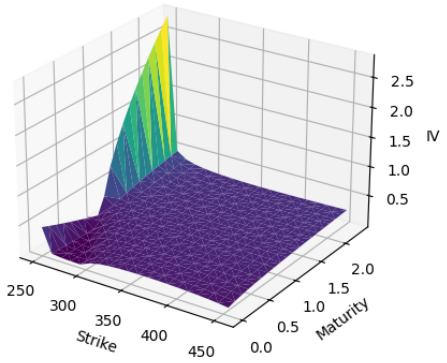

Fig. 3: Surface generated using a manually optimized SABR model, with all data points, resulting in an out-of-range peak.

Metric	Mean time	Standard Deviation
Execution time on entire dataset	229.1ms	41.2ms
Execution time per point	2ms	1ms

TABLE II: Model performance

For benchmarking the execution time, a small heatup of the model was performed, and the execution time was measured using the Python `perf_counter` method. Separate tests using a linear interpolation with the original points and with the implicit layer were performed to compare the generated surface's error. The results of the training and testing of the model are shown in Table I and Figure 2. The RMSE values for the test data show that the model was able to generalize well to the test data, with the lowest RMSE value for the ρ parameter. The error values were calculated by comparing the generated surface with the original points, as mentioned in the previous section.

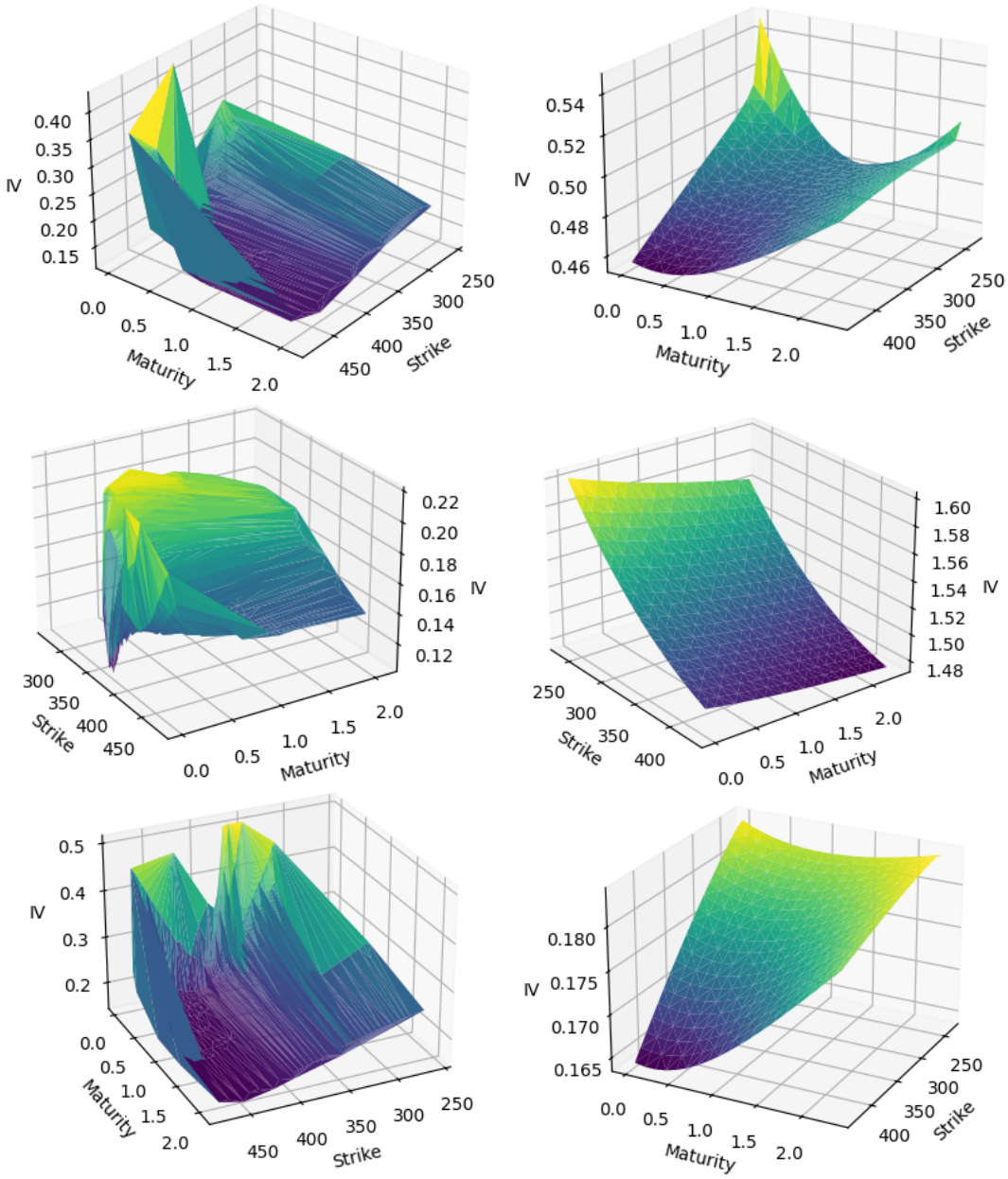

Fig. 4: Surfaces interpolated on test data using a linear interpolation model (left) compared to the SABR model fitted with the output of the implicit transformer architecture (right).

IV. Conclusion

A. Summary

This work presents an architecture taking as inspiration approaches from both Kim et al. [13] and Kolter et al. [15] for the task of adjusting the parameters of the SABR model using a neural network based on the transformer architecture (specifically the *encoder* element) with implicit deep equilibrium layers as the regression component. The combination of the SABR model with the transformer architecture for candidate selection was used to replicate the patterns of the daily implied volatility surface, generating a smooth surface that reduces arbitrage opportunities through the solution of a fixed-point problem with the implicit layers as outputs. The candidate scoring and selection process is performed automatically, instead of heuristically, and the fine-tuning of the model allows for new daily observations to be taken into account when adjusting the SABR parameters for the next trading day.

B. Implications and Future Work

The main focus of this work was to merge the approach using the transformer and the implicit layers to replace the manual selection of scores for candidates, as well to show

the possibility of future research focusing on High-Frequency Trading (HFT) options arbitrage strategies given the execution speed of the model shown in Table II. Additional market factors and input features are also an interesting improvement to the chosen approach and a promising extension in future research. Adapting the model to use extrinsic features besides the observed data, such as macroeconomic indicators, market indexes, and other financial instruments, could allow the model to adapt to changing market regimes and economic conditions, allowing for both exploitation and resilience to intraday arbitrage opportunities.

REFERENCES

[1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[2] Charles G Broyden. The convergence of a class of double-rank minimization algorithms 1. general considerations. *IMA Journal of Applied Mathematics*, 6(1):76–90, 1970.

[3] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. 2019. URL https://arxiv.org/abs/1806.07366.

[4] Roger Fletcher. A new approach to variable metric algorithms. *Computer Journal*, 13(3):317–322, 1970.

[5] Donald Goldfarb. A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109):23–26, 1970.

[6] Patrick Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Managing smile risk. *Wilmott Magazine*, 1:84–108, 01 2002.

[7] Patrick Hagan, Andrew Lesniewski, and Diana Woodward. *Probability Distribution in the SABR Model of Stochastic Volatility*, volume 110, pages 1–35. 06 2015. ISBN 978-3-319-11604-4. doi: 10.1007/978-3-319-11605-1_1.

[8] Patrick S. Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Arbitrage-free sabr. *Wilmott*, 2014(69):60–75, 2014. doi: https://doi.org/10.1002/wilm.10290. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/wilm.10290.

[9] Cristian Homescu. Implied volatility surface: Construction methodologies and characteristics, 2011.

[10] Richard Jordan and Charles Tier. Asymptotic approximations to cev and sabr models. *SSRN Electronic Journal*, pages 1–38, May 2011. Available at SSRN: https://ssrn.com/abstract=1850709 or http://dx.doi.org/10.2139/ssrn.1850709.

[11] Kenji Kawaguchi. On the theory of implicit deep learning: Global convergence with implicit layers. 2021. URL https://arxiv.org/abs/2102.07346.

[12] Bo-Hyun Kim, Daewon Lee, and Jaewook Lee. Local volatility function approximation using reconstructed radial basis function networks. In J Wang, Z Yi, JM Zurada, BL Lu, and H Yin, editors, *ADVANCES IN NEURAL NETWORKS - ISNN 2006, PT 3, PROCEEDINGS*, volume 3973 of *Lecture Notes in Computer Science*, pages 524–530, HEIDELBERGER PLATZ 3, D-14197 BERLIN, GERMANY, 2006. Univ Elect Sci & Technol China; Chinese Univ Hong Kong; Asia Pacific Neural Network Assembly; European Neural Network Soc; IEEE Circuits & Syst Soc; IEEE Computat Intelligence Soc; Int Neural Network Soc; Natl Nat Sci Fdn China; KC Wong Educ Fdn, Hong Kong, SPRINGER-VERLAG BERLIN. ISBN 3-540-34482-9. 3rd International Symposium on Neural Networks (ISNN 2006), Chengdu, PEOPLES R CHINA, MAY 28-31, 2006.

[13] Hyeonuk Kim, Kyunghyun Park, Junkee Jeon, Changhoon Song, Jungwoo Bae, Yongsik Kim, and Myungjoo Kang. Candidate point selection using a self-attention mechanism for generating a smooth volatility surface under the sabr model. *EXPERT SYSTEMS WITH APPLICATIONS*, 173, JUL 1 2021. ISSN 0957-4174. doi: 10.1016/j.eswa.2021.114640.

[14] Soohan Kim, Seok-Bae Yun, Hyeong-Ohk Bae, Muhyun Lee, and Youngjoon Hong. Physics-informed convolutional transformer for predicting volatility surface. *QUANTITATIVE FINANCE*, 24(2):203–220, JAN 9 2024. ISSN 1469-7688. doi: 10.1080/14697688.2023.2294799.

[15] Zico Kolter, David Duvenaud, and Matt Johnson. Deep implicit layers - neural odes, deep equilibrium models, and beyond, 2020. NeurIPS 2020 tutorial. Retrieved from https://implicit-layers-tutorial.org/.

[16] Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations. 2020. URL https://arxiv.org/abs/2001.01328.

[17] M. Ludkovski. Statistical machine learning for quantitative finance. *ANNUAL REVIEW OF STATISTICS AND ITS APPLICATION*, 10:271–295, 2023. ISSN 2326-8298. doi: 10.1146/annurev-statistics-032921-042409.

[18] Stefano Massaroli, Michael Poli, Jinkyoo Park, Atsushi Yamashita, and Hajime Asama. Dissecting neural odes. 2021. URL https://arxiv.org/abs/2002.08071.

[19] David F Shanno. Conditioning of quasi-newton methods for function minimization. *Mathematics of Computation*, 24(111):647–656, 1970.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2023. URL https://arxiv.org/abs/1706.03762.

[21] Homer F. Walker and Peng Ni. Anderson acceleration for fixed-point iterations. *SIAM Journal on Numerical Analysis*, 49(4):1715–1735, 2011. doi: 10.1137/10078356X.

[22] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. 2023. URL https://arxiv.org/abs/2202.07125.